Package 'skiLift'

October 12, 2026

Title DBI Interface to 'Snowflake'

Version 0.2.1

Description A DBI-compliant interface to Snowflake (<https://www.snowflake.com/en/>). Connects via Snowflake's SQL API v2 (HTTPS/JSON) as the universal baseline, with no dependency on ODBC, JDBC, or Python. Supports key-pair JWT, programmatic access token (PAT), OAuth, and session token authentication. Optionally uses ADBC (Arrow Database Connectivity) as an accelerator for bulk reads (Arrow-native) and writes (PUT + COPY INTO), activated automatically when adbcsnowflake is installed.

License Apache License 2.0 | file LICENSE

URL https://github.com/posit-dev/skiLift

BugReports https://github.com/posit-dev/skiLift/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 4.1.0)

Imports DBI (>= 1.2.0),
methods,
httr2 (>= 1.0.0),
jsonlite,
rlang (>= 1.0.0),
cli

Suggests openssl,
jose,
RcppTOML,
RcppSimdJson,
nanoarrow (>= 0.3.0),
adbcdrivermanager,
adbcsnowflake,
dbplyr (>= 2.3.0),
dplyr,
duckdb,
blob (>= 1.2.0),
hms (>= 1.0.0),
bit64,
reticulate,

snowflakeauth,
httpuv,
DBItest (>= 1.7.0),
testthat (>= 3.0.0),
mockr,
withr,
knitr,
rmarkdown

VignetteBuilder knitr

Config/testthat/edition 3

Contents

rcat . . . 2
rglimpse . . . 3
rprint . . . 3
rview . . . 4
sf_cache_in_duckdb . . . 4
sf_collect_to_duckdb . . . 5
sf_connect . . . 5
sf_duckdb_tables . . . 6
Snowflake . . . 6
SnowflakeConnection-class . . . 7
SnowflakeDriver-class . . . 10
SnowflakeResult-class . . . 11

`rcat` *Clean replacement for cat() in Workspace Notebooks*

Description

Builds the full string first, then outputs with `writeLines()` to avoid the extra line breaks that Workspace Notebooks add.

Usage

```
rcat(..., sep = "")
```

Arguments

`...`	Objects to concatenate and print.
`sep`	Separator between arguments. Default: "".

Value

Invisibly returns `NULL`.

rglimpse *Glimpse a data.frame structure*

Description

Glimpse a data.frame structure

Usage

```
rglimpse(x)
```

Arguments

x	A data.frame.

Value

Invisibly returns x.

rprint *Print a data.frame cleanly in Workspace Notebooks*

Description

Captures all output lines and writes them as a single block, avoiding the extra whitespace that Workspace Notebooks insert between lines.

Usage

```
rprint(x, width = 200L, ...)
```

Arguments

x	A data.frame or other printable object.
width	Integer. Print width in characters. Default: 200.
...	Additional arguments passed to print().

Value

Invisibly returns x.

rview *View the head of a data.frame cleanly*

Description

View the head of a data.frame cleanly

Usage

```
rview(x, n = 10L, width = 200L)
```

Arguments

x	A data.frame.
n	Integer. Number of rows to show. Default: 10.
width	Integer. Print width in characters. Default: 200.

Value

Invisibly returns the first n rows.

sf_cache_in_duckdb *Cache a Snowflake query result in a local DuckDB table*

Description

Executes sql against Snowflake (using the best available backend – ADBC if installed, REST API v2 otherwise) and writes the result into a DuckDB table for fast local analysis with dplyr / dbplyr.

Usage

```
sf_cache_in_duckdb(sf_con, sql, table_name, duck_con = NULL, overwrite = TRUE)
```

Arguments

sf_con	A SnowflakeConnection object.
sql	Character SQL query to run against Snowflake.
table_name	Character name for the DuckDB table.
duck_con	An existing DuckDB connection, or NULL to create one.
overwrite	Logical; overwrite if the table already exists? Default TRUE.

Value

The DuckDB connection (invisibly). If duck_con was NULL, the caller is responsible for disconnecting it.

sf_collect_to_duckdb *Materialise a* dplyr *lazy table from Snowflake into DuckDB*

Description

Takes a tbl backed by a Snowflake connection, collects the underlying SQL, executes it against Snowflake, and writes the result into DuckDB for continued dplyr analysis.

Usage

```
sf_collect_to_duckdb(.tbl, table_name, duck_con = NULL, overwrite = TRUE)
```

Arguments

.tbl	A tbl_lazy backed by a SnowflakeConnection.
table_name	Character name for the DuckDB table.
duck_con	An existing DuckDB connection, or NULL to create one.
overwrite	Logical; overwrite if the table already exists? Default TRUE.

Value

A tbl backed by DuckDB, ready for further dplyr ops.

sf_connect *Connect to Snowflake using a profile name*

Description

Convenience wrapper around dbConnect(Snowflake(), name = ...).

Usage

```
sf_connect(name = NULL, ...)
```

Arguments

name	Profile name from connections.toml.
...	Additional arguments passed to dbConnect.

Value

A SnowflakeConnection.

sf_duckdb_tables *List tables cached in a DuckDB connection*

Description

List tables cached in a DuckDB connection

Usage

```
sf_duckdb_tables(duck_con)
```

Arguments

duck_con	A DuckDB connection.

Value

Character vector of table names.

Snowflake *Create a SnowflakeDriver instance*

Description

Create a SnowflakeDriver instance

Usage

```
Snowflake()
```

Value

A SnowflakeDriver singleton.

Examples

```
## Not run:
drv <- Snowflake()
con <- dbConnect(drv, account = "myaccount", name = "default")

## End(Not run)
```

SnowflakeConnection-class *SnowflakeConnection*

Description

An S4 class representing a connection to Snowflake via the SQL API v2. Created by `DBI::dbConnect()` with a `Snowflake()` driver.

Usage

```
## S4 method for signature 'SnowflakeConnection'
show(object)

## S4 method for signature 'SnowflakeConnection'
dbIsValid(dbObj, ...)

## S4 method for signature 'SnowflakeConnection'
dbDisconnect(conn, ...)

## S4 method for signature 'SnowflakeConnection'
dbGetInfo(dbObj, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbSendQuery(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbSendStatement(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbGetQuery(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbExecute(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbGetQueryArrow(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbSendQueryArrow(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection'
dbListTables(conn, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbExistsTable(conn, name, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbListFields(conn, name, ...)

## S4 method for signature 'SnowflakeConnection'
```

```
dbListObjects(conn, prefix = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbReadTable(conn, name, ...)

## S4 method for signature 'SnowflakeConnection,character,data.frame'
dbWriteTable(
  conn,
  name,
  value,
  overwrite = FALSE,
  append = FALSE,
  row.names = FALSE,
  ...
)

## S4 method for signature 'SnowflakeConnection,character'
dbCreateTable(conn, name, fields, ..., row.names = NULL, temporary = FALSE)

## S4 method for signature 'SnowflakeConnection,character'
dbAppendTable(conn, name, value, ..., row.names = NULL)

## S4 method for signature 'SnowflakeConnection,character'
dbRemoveTable(conn, name, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbQuoteIdentifier(conn, x, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbQuoteString(conn, x, ...)

## S4 method for signature 'SnowflakeConnection,SQL'
dbQuoteString(conn, x, ...)

## S4 method for signature 'SnowflakeConnection,SQL'
dbQuoteIdentifier(conn, x, ...)

## S4 method for signature 'SnowflakeConnection,SQL'
dbUnquoteIdentifier(conn, x, ...)

## S4 method for signature 'SnowflakeConnection'
dbQuoteLiteral(conn, x, ...)

## S4 method for signature 'SnowflakeConnection'
dbDataType(dbObj, obj, ...)

## S4 method for signature 'SnowflakeConnection'
sqlData(con, value, row.names = FALSE, ...)

## S4 method for signature 'SnowflakeConnection'
dbBegin(conn, ...)
```

```
## S4 method for signature 'SnowflakeConnection'
dbCommit(conn, ...)

## S4 method for signature 'SnowflakeConnection'
dbRollback(conn, ...)

## S4 method for signature 'SnowflakeConnection'
dbWithTransaction(conn, code, ...)
```

Arguments

object	A SnowflakeConnection object (for show).
dbObj	A SnowflakeConnection object.
...	Additional arguments (currently ignored).
conn	A SnowflakeConnection object.
statement	SQL statement string.
params	Optional parameter list for parameterized queries.
name	Table name (character).
prefix	An Id object indicating the hierarchy level to list (NULL for databases, 1-component for schemas, 2-component for tables).
value	A data.frame.
overwrite	Logical. Drop existing table first?
append	Logical. Append to existing table?
row.names	Ignored (always FALSE for Snowflake).
fields	A data.frame or named character vector of column types.
temporary	Logical. Create a temporary table?
x	Character to quote.
obj	An R object to map to a Snowflake SQL type.
con	A SnowflakeConnection object.
code	Code to execute within the transaction.

Slots

account Account identifier.

user Username.

database Default database.

schema Default schema.

warehouse Default warehouse.

role Default role.

.auth Auth list (type, token, token_type, plus params for refresh).

.state Environment holding mutable state (valid, in_transaction, session_info, and the live auth token cache – token, token_mtime).

SnowflakeDriver-class *SnowflakeDriver*

Description

An S4 class representing the Snowflake DBI driver.

Usage

```
## S4 method for signature 'SnowflakeDriver'
dbGetInfo(dbObj, ...)

## S4 method for signature 'SnowflakeDriver'
dbIsValid(dbObj, ...)

## S4 method for signature 'SnowflakeDriver'
dbUnloadDriver(drv, ...)

## S4 method for signature 'SnowflakeDriver'
show(object)

## S4 method for signature 'SnowflakeDriver'
dbDataType(dbObj, obj, ...)

## S4 method for signature 'SnowflakeDriver'
dbConnect(
  drv,
  account = NULL,
  user = NULL,
  token = NULL,
  private_key_path = NULL,
  authenticator = NULL,
  database = "",
  schema = "",
  warehouse = "",
  role = "",
  name = NULL,
  ...
)

## S4 method for signature 'SnowflakeDriver'
dbCanConnect(drv, ...)
```

Arguments

dbObj	A SnowflakeDriver object.
...	Additional arguments (ignored).
drv	A SnowflakeDriver, or missing (uses default).
object	A SnowflakeDriver object (for show).
obj	An R object to map to a Snowflake SQL type.

account	Snowflake account identifier (e.g. "myaccount").
user	Snowflake username.
token	Explicit bearer token (PAT or session token).
private_key_path	Path to PEM private key for JWT auth.
authenticator	Auth method ("SNOWFLAKE_JWT", "OAUTH", etc.).
database	Default database.
schema	Default schema.
warehouse	Default warehouse.
role	Default role.
name	Profile name from connections.toml.

Connecting with no arguments

`dbConnect(Snowflake())` can be called with **no arguments at all** inside the Posit Team Native App (Native App) or Posit Workbench. Both write their own `connections.toml` profile before your code runs; when `account` is `NULL`, `dbConnect()` reads that profile automatically. The connection authenticates as the Snowflake user who launched the Native App session – not a shared service account – in that user's default role, warehouse, and database, unless you override any of `account`, `user`, `warehouse`, `database`, `schema`, or `role` explicitly. An explicit argument always wins over the profile; the profile always wins over an empty default.

`SnowflakeResult-class` *SnowflakeResult*

Description

An S4 class representing the result of a query against Snowflake. Created by `DBI::dbSendQuery()` or `DBI::dbSendStatement()`.

Usage

```
## S4 method for signature 'SnowflakeResult'
show(object)

## S4 method for signature 'SnowflakeResult'
dbIsValid(dbObj, ...)

## S4 method for signature 'SnowflakeResult'
dbClearResult(res, ...)

## S4 method for signature 'SnowflakeResult'
dbFetch(res, n = -1, ...)

## S4 method for signature 'SnowflakeResult'
dbHasCompleted(res, ...)

## S4 method for signature 'SnowflakeResult'
```

```
dbFetchArrow(res, ...)

## S4 method for signature 'SnowflakeResult'
dbFetchArrowChunk(res, ...)

## S4 method for signature 'SnowflakeResult'
dbGetRowCount(res, ...)

## S4 method for signature 'SnowflakeResult'
dbGetRowsAffected(res, ...)

## S4 method for signature 'SnowflakeResult'
dbGetStatement(res, ...)

## S4 method for signature 'SnowflakeResult'
dbColumnInfo(res, ...)

## S4 method for signature 'SnowflakeResult'
dbGetInfo(dbObj, ...)

## S4 method for signature 'SnowflakeResult'
dbBind(res, params, ...)
```

Arguments

`object`	A SnowflakeResult object (for show).
`dbObj`	A SnowflakeResult object.
`...`	Additional arguments (currently ignored).
`res`	A SnowflakeResult object.
`n`	Number of rows to fetch. -1 (default) fetches all.
`params`	A list of parameter values for parameterized queries.

Slots

`connection` The parent SnowflakeConnection.

`statement` The SQL statement string.

`.resp_body` The raw JSON response from the first partition.

`.meta` Parsed metadata (columns, partitions, handle).

`.state` Environment holding mutable state (fetched, partition, valid, etc.).